Embedded Data Structure
(Lecture 02)

Revisit C Programming (1)

2025. 3. 7

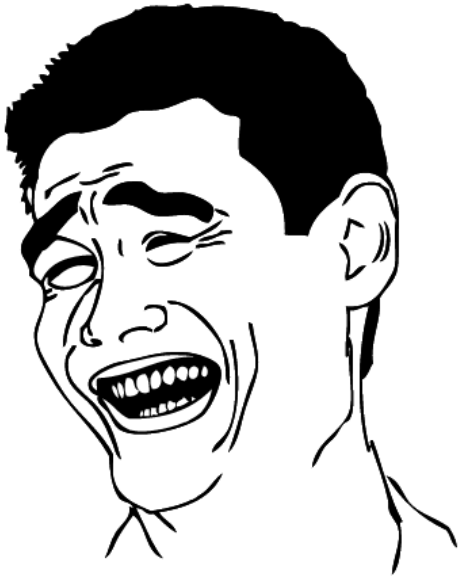
Prof. Wonjun Kim

School of Electrical and Electronics Engineering

Review of the Last Class

- Data is a key point in the future world
- C plays an important role to understand "handling data"
  - Understanding of data access in the system
  - Understanding of memory structure in the system

But hey, are you familiar with C programming ?



Quiz

- **Solve the following examples**

```
#include <stdio.h>

void main()
{
    int a = 9;
    int b = 39;
    int *pt;

    pt = &a;
    b = *pt;

    printf("b value = %d₩n", b);
}
```

```
#include <stdio.h>

void main()
{
    int a = 9;
    int b = &a;
    int *pt;

    pt = b;

    printf("result = %f₩n", *pt);
}
```

➔ What is the result for each example ?

Array (1/6)

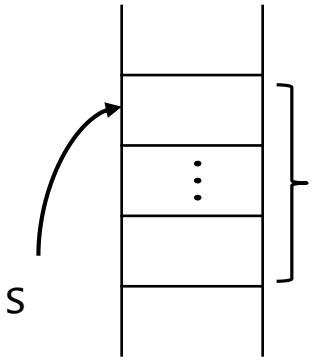
- **Necessity of array : introduce the concept of memory**
  - **How about 100 scores or 1000 scores ?**

```
int s0, s1, s2, s3, s4, s5, s6, s7, s8, s9;  // for 10 scores
```

```
int s[10];
```

s[0]	s[1]	s[2]	s[3]	s[4]	s[5]	s[6]	s[7]	s[8]	s[9]

- Array uses the memory space with its size
- You can manage(handle) a large amount of data



Array (2/6)

▪ Array declaration

- **Using the "[ ]" with definition of its size**

```
int score[10];
```

Type Name Size

<u>For example :</u>

```
int   score[100];
float   average[39];
char filename[200];
```

Consider following cases

① int score[10];

② [ ] int size = 10;

int score[size];

➡ Can you expect the other one ?

③ #define size 10

int score[size];

➡ **Array is a "static" data structure ! (Size should be fixed)**

Array (3/6)

▪ Access to the array

- **Be careful handling the index of the array**

  - Start with "0", e.g., score[0] = 95; / score[0] = score[9];

```
#include  <stdio.h>

void main()
{
   int score[5];

   score[0] = 95;
   score[1] = 95;
   score[2] = 95;
   score[3] = 95;
   score[4] = 95;
}
```

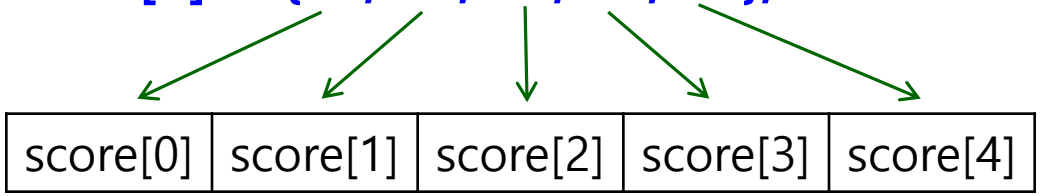
How about score[5] ?

```
#include  <stdio.h>
#define SIZE 5

void main()
{
   int j;
   int score[SIZE];

   for(j = [   ] ; j < [   ] ; j++)
   {
      [                    ]
   }
}
```

Array (4/6)

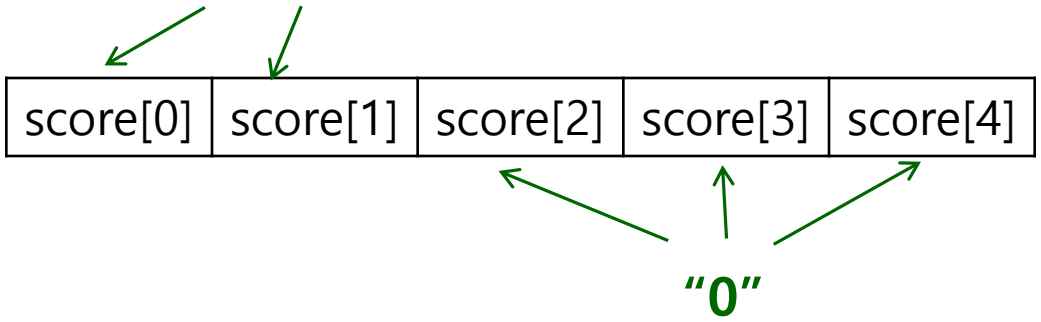
▪ Initialization of array

- **Basic formulation : int score[5] = {94, 87, 92, 75, 83};**



- How about the smaller number of values for initialization ?

  For example, int score[5] = { 94, 87};



✓ **How can you initialize your array as "0" ?**

✓ **Consider the case (int score[5];)**

Array (5/6)

▪ Computing the size of the array

- **It can be useful for the case using a lot of elements**

For example, int score[] = {31, 45, 67, 53, 99, 93, 82, 67, 78, 88, 85, 47};

You need to carefully count !

➡ **We use the "sizeof()" operator as shown below :**

```
#include  <stdio.h>

void main()
{
   int score[] = {97, 88, 83, 79, 65, 58, 92, 89, 77, 81, 60};
   int size;

   size = sizeof(score) / sizeof(score[0]);
   printf("size of array = %d₩n", size);
}
```

Array (6/6)

- **Array operations**
  - **Copy elements of the array**

```
int a[SIZE] = {1, 2, 3, 4, 5};
int b[SIZE];

b = a;
```

What does it mean ?

```
int a[SIZE] = {1, 2, 3, 4, 5};
int b[SIZE];

for(int j = 0; j<SIZE; j++)
   b[j] = a[j];
```

  - **Toy example : implement the following codes**

1) Initialize the array **b** with the reverse order of the array **a**

2) Store the results of multiplication between a and b in the array **c**

Pointer : Definition (1/2)

- **Interpret the following statement**
  - **If you can do that, that's all !**

```
int *p;
```

- **Pointer :**
- **Address operator &**

  : "&" represents the address of the variable (or function, structure, etc.)

  → int j = 9; printf("Address of j = %d₩n", &j);

Pointer : Definition (2/2)

- **Simple practice**
  - **Consider the following example codes**

```
int j = 9;
int *p = &j;
```

⬇

```
int j = 9;
int *p;
```

Fill the blank for the same operation

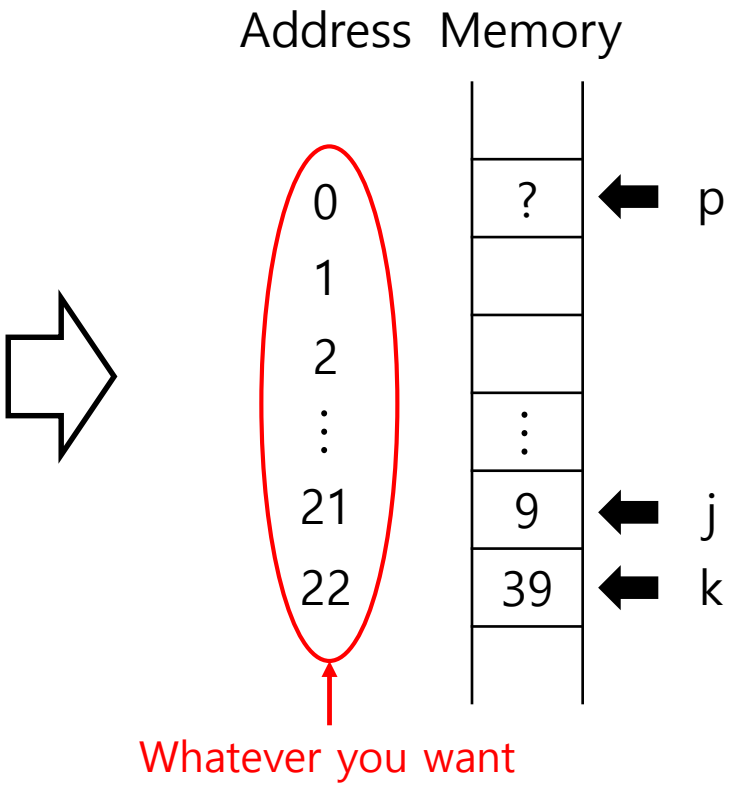
```
#include  <stdio.h>

void main()
{
    int j = 9;
    float f = 9.3;
    int *pj = NULL;
    int *pf = NULL;

    pj =

    pf =
}
```

➡ **Check the result using "printf"**

Dereferencing (1/2)

- **Value indicated by the pointer**
  - **Using "*", we can refer the value indicated by the pointer**
    - Always try to **draw** the procedure related to the pointer !

```
#include  <stdio.h>

void main()
{
    int j = 9;
    int *p = NULL;

    p = &j;

    printf("j = %d₩n", j);
    printf("p = %d₩n", p);
    printf("*p = %d₩n", *p);
}
```



Dereferencing (2/2)

- **Simple practice**
  - **Assign a specific value to the variable indicated by the pointer**

```
#include   <stdio.h>

void main()
{
    int x = 9, y = 39;
    int *p;

    p = &x;
    printf("*p = %d₩n", *p);

    p = &y;
    printf("*p = %d₩n", *p);
}
```

```
#include   <stdio.h>

void main()
{
    int j = 9;
    int *p;

    p = &j;
    printf("j = %d₩n", j);

    *p = 39;
    printf("j = %d₩n", j);
}
```

➡ Expect the print result of both example codes without coding

Some Notes on Pointers

KU KONKUK UNIVERSITY

- **Transform notations – cont'd**
  - **Do not use a specific value for the address**
    - CPU reserves some spaces for the system management
  - **Initialization of pointers is recommended**
    - Random assignment yields the address of the system memory sometimes

```
void main()
{
    int *p;

    *p = 100;
}
```

```
void main()
{
    int *p = 1000;

    *p = 123;
}
```

```
#include <stdio.h>

void main()
{
    int j;
    float *pf;

    pf = &j;
    *pf = 9.3;
}
```

➡ If needed, fix some errors !

Pointer Operations (1/4)

KU KONKUK UNIVERSITY

▪ Pointer operations : addition and subtraction

- **Adaptive access is available even though memory structure is linear**
  - For example, int *p and the value of p is 1,000 → how about p+1 ?

[Bytes for each data type]

Pointer type	Increased value
char	1
short	2
int	4
float	4
double	8

※ Memory unit : bytes

```
#include <stdio.h>

void main()
{
    char *pc;
    int *pi;
    float *pf;

    pc = (char *)1000;
    pi = (int *)1000;
    pf = (float *)1000;

    // Check pc+2, pi+2, pf+2 !
}
```

Why do we use it ?

Pointer Operations (2/4)

KU KONKUK UNIVERSITY

▪ Combination with the incremental operator

- **Discriminate following two operations : "*p++" and "(*p)++"**
  - Think about the precedence of operators

```
#include <stdio.h>

void main() {
    int j = 9;
    int *p = &j;

    printf("j = %d,  p = %d₩n", j, p);
    (*p)++;
    printf("j = %d,  p = %d₩n", j, p);

    printf("j = %d,  p = %d₩n", j, p);
    *p++;
    printf("j = %d,  p = %d₩n", j, p);
}
```

What does it mean ?

What does it mean ?

※ Think about the incremental operators of pointers (system stability)

Pointer Operations (3/4)

- **Simple practice**
  - **Expect the results of the following codes without coding**

```c
#include <stdio.h>

void main()
{
    int a = 9;
    int b = 39;
    int *pt;

    pt = &a;
    b = *pt;

    printf("b value = %d₩n", b);
}
```

```c
#include <stdio.h>

void main()
{
    int a = 9;
    int *pt1, *pt2;

    pt1 = &a;
    pt2 = &a+1;
    *pt2 = 39;

    printf("%d₩n", *(pt1+1));
    printf("%d₩n", *(pt1)+1);
}
```

➡ Draw the procedure of pointer operations on your paper !

Pointer Operations (4/4)

- **Pointer casting (just intro)**
  - **Why ? memory management in the embedded system**

Basic formulation

```
float f;
float *pf = &f;
int *pi;

pi = (int *)pf;
```

Think about meaning of this statement !

What is *pi ?

```
#include <stdio.h>

void main()
{
    char buffer[8];
    int *pi;

    pi = (int *)buffer;
    *pi = 12;
    *(pi+1) = 34;
    printf("%d₩n%d₩n",*pi, *(pi+1));
}
```

How to print the same value using "buffer" ?

Summary

- **Basic concept of array and pointer**
  - **Array and pointer are same except for …**
  - **Remember usage of array and pointer**
  - **Remember the definition of pointer**

    : Pointer is a local variable containing the address of the variable